R3.07

SQL dans un langage de programmation

Transactions

Problèmes de cohérence et transactions

Un SGBD doit pouvoir supporter :

- plusieurs utilisateurs l'utilisant en parallèle.
- effectuant des opérations d'écriture et de lecture.
- tout en garantissant la cohérence des données

Une transaction est un ensemble d'ordres SQL indivisibles, faisant passer la base de données d'un état cohérent à un autre état cohérent en une seule étape.

Propriétés ACID

Atomicité Une transaction réussi ssi toutes ses opérations réussissent.

Cohérence Une transaction terminée laisse la base dans un état cohérent où les données sont intègres.

Isolation Les transactions doivent être rendues indépendantes les unes des autres.

Durabilité Les effets d'uns transaction terminée sont persistants.

Opérations COMMIT et ROLLBACK

ROLLBACK *annule* entièrement une transaction : toutes les modifications depuis le début de la transaction sont alors défaites.

COMMIT *valide* entièrement une transaction : les modifications deviennent définitives et visibles à tous les utilisateurs.

SAVEPOINT point de contrôle, état de la base où l'on pourra revenir plus tard.

A noter

- En cours de transaction, *seul l'utilisateur ayant effectué les modifications les voit*.
- En cas de fin anormale d'une tâche utilisateur, il y a automatiquement `ROLLBACK` des transactions non terminées.
- Une transaction commence (implicitement) à la première opération SQL rencontrée et dès qu'une transaction est terminée.
- Les commandes de définition de données sont automatiquement commitées (*auto-commit*) et valident donc les ordres précédents.
- Un mécanisme de verrouillage permet de gérer les conflits d'accès parallèle.

Exemple de transaction

Créer une table `test` : `CREATE TABLE test (a integer);`

Question 1

1. Commencer une transaction : `BEGIN TRANSACTION;`
2. Insérer les deux valeurs 1 et 2 dans la table : `INSERT INTO ...`
3. Valider la transaction : `COMMIT;`
4. Vérifier que les valeurs sont bien dans la table : `SELECT * FROM ...`

Exemple de transaction

Créer une table `test` : `CREATE TABLE test (a integer);`

Question 1

1. Commencer une transaction : `BEGIN TRANSACTION;`
2. Insérer les deux valeurs 1 et 2 dans la table : `INSERT INTO ...`
3. Valider la transaction : `COMMIT;`
4. Vérifier que les valeurs sont bien dans la table : `SELECT * FROM ...`

Solution

```sql
BEGIN TRANSACTION;
INSERT INTO test(a) VALUES (1);
INSERT INTO test(a) VALUES (2);
COMMIT;
SELECT * FROM test;

```

```
 a
---
 1
 2
```

Exemple de transaction

Question 2

1. Commencer une transaction : `BEGIN TRANSACTION;`
2. Insérer les deux valeurs 3 et 4 dans la table : `INSERT INTO ...`
3. Annuler la transaction : `ROLLBACK;`
4. Vérifier que les valeurs ne sont pas dans la table : `SELECT * FROM ...`

Exemple de transaction

Question 2

1. Commencer une transaction : `BEGIN TRANSACTION;`
2. Insérer les deux valeurs 3 et 4 dans la table : `INSERT INTO ...`
3. Annuler la transaction : `ROLLBACK;`
4. Vérifier que les valeurs ne sont pas dans la table : `SELECT * FROM ...`

Solution

```sql
BEGIN TRANSACTION;
INSERT INTO test(a) VALUES (3);
INSERT INTO test(a) VALUES (4);
ROLLBACK;
SELECT * FROM test;

```

```
 a
---
 1
 2
```

Simulation d'une panne

Créer une table `test` : `CREATE TABLE test (a integer);`

Question

1. Commencer une transaction : `BEGIN TRANSACTION;`
2. Insérer les deux valeurs 1 et 2 dans la table : `INSERT INTO ...`
3. Vérifier que les valeurs sont bien dans la table : `SELECT * FROM ...`
4. Simuler un crash en fermant le terminal : `ROLLBACK` du système
5. Se reconnecter et vérifier que les valeurs *ne sont plus* dans la table : `SELECT * FROM ...`

Simulation d'une panne

Solution

```
1 BEGIN TRANSACTION;
2 INSERT INTO test(a) VALUES (1);
3 INSERT INTO test(a) VALUES (2);
4 SELECT * FROM test;
5
6
7
```

```
1  a
2 ---
3  1
4  2
```

```
1 -- SIMULATION DE CRASH
2 SELECT * FROM test;
```

```
1  a
2 ---
3 (0 rows)
4
```

Simulation d'accès concurrents

Se connecter deux fois à une même base de données dans deux terminaux (term1 et term2).
Créer une table `test` : `CREATE TABLE test (a integer);`

Question 1

1. term1 Insérer une valeur : `INSERT ...` (`COMMIT` implicite)
2. term2 Vérifier que la valeur est visible : `SELECT ...`

Simulation d'accès concurrents

Se connecter deux fois à une même base de données dans deux terminaux (term1 et term2).
Créer une table `test` : `CREATE TABLE test (a integer);`

Question 1

1. term1 Insérer une valeur : `INSERT ...` (`COMMIT` implicite)
2. term2 Vérifier que la valeur est visible : `SELECT ...`

Solution

```
1 -- term1
2 INSERT INTO test(a) VALUES (1);
```

```
1 -- term2
2 SELECT * FROM test;
```

```
1  a
2 ---
3  1
```

Simulation d'accès concurrents

Question 2

1. term1 Commencer une transaction : `BEGIN TRANSACTION;`
2. term1 Insérer la valeur 2 dans la table : `INSERT ...`
3. term1 Vérifier que la valeur est visible : `SELECT * FROM ...`
4. term2 Vérifier que la valeur n'est pas visible : `SELECT * FROM ...`
5. term1 Valider la transaction : `COMMIT;`
6. term2 Vérifier que la valeur est visible : `SELECT * FROM ...`

Simulation d'accès concurrents

Solution

```
1 -- term1
2 BEGIN TRANSACTION;
3 INSERT INTO test(a) VALUES (2);
4 SELECT * FROM test;
5
```

```
1  a
2 ---
3  1
4  2
```

```
1 -- term2
2 SELECT * FROM test;
```

```
1  a
2 ---
3  1
```

```
1 -- term1
2 COMMIT;
3
```

```
1 -- term2
2 SELECT * FROM test;
```

```
1  a
2 ---
3  1
4  2
```

Simulation d'accès concurrents

Question 3

1. term1 Commencer une transaction : `BEGIN TRANSACTION;`
2. term1 Exécuter une mise à jour (multiplier toutes les valeurs par 2) : `UPDATE ...`
3. term2 Exécuter une mise à jour concurrente (multiplier toutes les valeurs par 3) : `UPDATE ...`
4. term2 Observer la mise en attente
5. term1 Valider la transaction : `COMMIT;`
6. term2 Vérifier le déblocage et que les deux mises à jour ont bien été effectuées (a multiplié les valeurs par 6) : `SELECT ...`
7. term1 Vérifier que les deux mises à jour ont bien été effectuées (a multiplié les valeurs par 6) : `SELECT ...`

Simulation d'accès concurrents

Solution

```
1 -- term1
2 BEGIN TRANSACTION;
3 UPDATE test SET a=a*2;
4
```

```
1 -- term2
2 UPDATE test SET a=a*3;
3 -- attente...
```

```
1 -- term1
2 COMMIT;
```

```
1 -- term2
2 SELECT * FROM test;
```

```
1  a
2 ----
3   6
4  12
```

```
1 -- term1
2 SELECT * FROM test;
```

```
1  a
2 ----
3   6
4  12
```

Problèmes soulevés par la concurrence

1. Perte de mise à jour

Temps	Transaction A	Transaction B
t1	LIRE T	
t2	...	LIRE T
t3	UPDATE T	...
t4	...	UPDATE T
t5	COMMIT	...
t4		COMMIT

Problème de la perte de mise à jour du tuple T par la transaction A

Les transactions A et B accèdent au même tuple T ayant la même valeur respectivement à l'instant t1 et l'instant t2.
Elles modifient chacune la valeur de T.
Les modifications effectuées par A seront perdues puisqu'elle avait lu T avant sa modification par B.

Problèmes soulevés par la concurrence

1. Perte de mise à jour : exemple

Temps	A : Ajouter 100	B : Ajouter 10
t1	LIRE COMPTE C=1000	
t2	...	LIRE COMPTE C=1000
t3	UPDATE COMPTE C=C+100=1100	...
t4	...	UPDATE COMPTE C=C+10=1010
t5	COMMIT C=1100	...
t6		COMMIT C=1010

Doucle crédit d'un compte bancaire

Dans cet exemple, le compte bancaire vaut 1010 à la fin des deux transactions à la place de 1110.

Problèmes soulevés par la concurrence

2. Accès à des données non validées

Temps	Transaction A	Transaction B
t1		UPDATE T
t2	LIRE T	...
t3		ROLLBACK

Problème de la lecture impropre du tuple T par la transaction A

La transaction A accède au tuple T qui a été modifié par la transaction B. La transaction B annule sa modification et A a donc accédé à une valeur qui n'aurait jamais dû exister (virtuelle) de T.
Pire, la transaction A pourrait faire une mise à jour de T après l'annulation par B, cette mise à jour inclurait la valeur avant annulation par B (et donc reviendrait à annuler l'annulation de B).

Problèmes soulevés par la concurrence

2. Accès à des données non validées : exemple

Temps	A : Ajouter 10	B : Ajouter 100 (erreur)
t1		LIRE COMPTE C=1000
t2		UPDATE COMPTE C=C+100=1100
t3	LIRE COMPTE C=1100	...
t4	...	ROLLBACK C=1000
t5	UPDATE C C=C+10=1110	
t6	COMMIT C=1110	

Annulation de crédit sur le compte bancaire C

Dans cet exemple, le compte bancaire vaut 1110 à la fin des deux transactions à la place de 1010.

Problèmes soulevés par la concurrence

3. Lecture incohérente

Temps	Transaction A	Transaction B
t1	LIRE T	
t2	...	UPDATE T
t3	...	COMMIT
t4	LIRE T	

Problème de la lecture non reproductible du tuple T par la transaction A

Si au cours d'une même transaction A accède deux fois à la valeur d'un tuple alors que ce dernier est, entre les deux, modifié par une autre transaction B, alors la lecture de A est inconsistante. Ceci peut entraîner des incohérences, par exemple si un calcul est en train d'être fait sur des valeurs par ailleurs en train d'être mises à jour par d'autres transactions.

Le verrouillage

Définition

La technique du verrouillage est une solution simple à la gestion de la concurrence d'accès. Une transaction peut poser un verrou (ou *lock*) sur un objet (typiquement un tuple) afin de rendre cet objet inaccessible aux autres transactions.

Verrou partagé S

Un verrou partagé, noté S, est posé par une transaction sur un objet lors d'un accès en *lecture* sur cet objet.
Un verrou partagé interdit aux autres transactions de poser un verrou exclusif sur cet objet et donc d'y accéder en écriture.

Le verrouillage

Verrou exclusif X

Un verrou exclusif, noté X, est posé par une transaction sur un objet lors d'un accès en *écriture* sur cet objet.
Un verrou exclusif interdit aux autres transactions de poser tout autre verrou (partagé ou exclusif) sur cet objet et donc d'y accéder (ni en lecture, ni en écriture).

Verrous S multiples

Un même objet peut être verrouillé de façon partagée par plusieurs transactions en même temps. Il sera impossible de poser un verrou exclusif sur cet objet tant qu'au moins une transaction disposera d'un verrou S sur cet objet.

Règles de verrouillage

Soit la transaction A voulant poser un verrou S sur un objet O

1. Si O n'est pas verrouillé alors A peut poser un verrou S
2. Si O dispose déjà d'un ou plusieurs verrous S alors A peut poser un verrou S
3. Si O dispose déjà d'un verrou X alors A ne peut pas poser de verrou S

Soit la transaction A voulant poser un verrou X sur un objet O

1. Si O n'est pas verrouillé alors A peut poser un verrou X
2. Si O dispose déjà d'un ou plusieurs verrous S ou d'un verrou X alors A ne peut pas poser de verrou X

	Verrou X présent	Verrou(s) S présent(s)	Pas de verrou présent
Verrou X demandé	Demande refusée	Demande refusée	Demande accordée
Verrou S demandé	Demande refusée	Demande accordée	Demande accordée

Matrice des règles de verrouillage

Le verrouillage

Le déverrouillage

Lorsqu'une transaction se termine (`COMMIT` ou `ROLLBACK`), elle libère tous les verrous qu'elle a posés.

L'inter-blocage

L'inter-blocage est le phénomène qui apparaît quand deux transactions se bloquent mutuellement par des verrous posés sur les données.
Ces verrous empêchent chacune des transactions de se terminer et donc de libérer les verrous qui bloquent l'autre transaction. Un processus d'attente sans fin s'enclenche alors.
Les situations d'inter-blocage sont détectées par les SGBD et gérées, en annulant l'une, l'autre ou les deux transactions, par un ROLLBACK système.

Solution aux problèmes soulevés par la concurrence

Le principe du verrouillage permet d'apporter une solution aux trois problèmes classiques soulevés par les accès concurrents aux données par les transactions.

1. Perte de mise à jour

Temps	Transaction A	Transaction B
t1	LIRE T Verrou S	
t2	...	LIRE T Verrou S
t3	UPDATE T Attente...	...
t4	... Attente...	UPDATE T Attente...
...	Inter-blocage	

Problème de la perte de mise à jour du tuple T par la transaction A

Le problème de perte de mise à jour est réglé, mais soulève ici un autre problème, celui de l'inter-blocage.

Solution aux problèmes soulevés par la concurrence

2. Accès à des données non validées

Temps	Transaction A	Transaction B
t1		UPDATE T Verrou X
t2	LIRE T Attente...	...
t3		ROLLBACK Libération du verrou X
t4	Verrou S	

Problème de la lecture impropre du tuple T par la transaction A

Solution aux problèmes soulevés par la concurrence

3. Lecture incohérente

Temps	Transaction A	Transaction B
t1	LIRE T Verrou S	
t2	...	UPDATE T Attente...
t3	LIRE T Verrous S	...
...	... libération des verrous ...	... reprise de la transaction ...

Problème de la lecture non reproductible du tuple T par la transaction A